



Cedar Initiative by NoxusDynamics LLP

PROJECT REPORT

on

Crop Recommendation System

TEAMNCD-10

Submitted by:

Aswani

Thamanna

Bharat

Sreedevi

September 2026

Crop Recommendation System

Machine Learning – Random Forest Classifier / Streamlit Web Interface

Project Repository: <https://github.com/noxusdynamics/NCD-10.git>

1. Team & Role Distribution

The project was executed by a four-member team, with each member responsible for a distinct crop's data as well as one stage of the ML pipeline.

Name	Crop Data Assigned	Primary Responsibility
Aswani (Project Lead)	Papaya	Data preprocessing; GitHub repo ownership & merging; PR review/merge, end-to-end pipeline testing, presentation slides
Thamanna	Coconut	Train/Test split (80/20) + Random Forest model training
Bharat	Rice	Model evaluation — accuracy, confusion matrix, classification report
Sreedevi	Banana	Model persistence (joblib) + Streamlit interface

2. Introduction & Objective

The Crop Recommendation System is a supervised machine learning project that recommends the most suitable crop for a given set of soil and climate conditions. Instead of a farmer manually guessing which crop fits their land, the system takes measurable inputs — Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, soil pH, and rainfall — and returns a predicted crop label.

The team narrowed the scope to four crops for this iteration — banana, coconut, papaya, and rice — chosen because they have visibly distinct soil and climate requirements, making them a good starting point for a classifier before scaling to a larger crop set.

3. Methodology / Workflow

The project followed a 10-step plan drafted at the start (Steps 1– 3 were completed collaboratively before work was split; Steps 4 onward were divided by team member):

Step	Description	Owner
1	Define problem & finalize target crops	Whole team
2	Gather dataset (Kaggle – Crop Recommendation Dataset)	Whole team
3	Clean & preprocess data (remove nulls/duplicates, scale features)	Aswani
4	Train/test split (80/20, stratified) + train Random Forest	Thamanna
5	Evaluate model (accuracy, confusion matrix, classification report)	Bharat
6	Save model + scaler; build Streamlit interface	Sreedevi
7	GitHub PR review/merge, pipeline testing, slide preparation	Aswani

3.1 Steps 1– 3: Problem Definition, Dataset, Preprocessing

- Crops finalized: banana, coconut, papaya, rice — selected for having clearly different soil/climate profiles.
- Dataset source: Kaggle's "Crop Recommendation Dataset," which already contains the required columns (N, P, K, temperature, humidity, pH, rainfall, label).
- Rows were filtered down to the four target crops and combined into a single working file, combined_dataset.csv.
- Preprocessing (owned by the project lead): features were separated from the label and passed through scikit-learn's StandardScaler so that all features sit on a comparable numeric range before training.

```
import pandas as pd
from sklearn.preprocessing import StandardScaler

data = pd.read_csv("combined_dataset.csv")
X = data[['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']]
y = data['label']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

3.2 Step 4: Train/Test Split & Model Training— Thamanna

The scaled dataset (488 rows × 7 features) was split into an 80/20 train-test partition using a stratified split, ensuring all four crops are proportionally represented in both sets. A Random Forest Classifier (100 trees) was then trained on the training partition.

```

import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

data = pd.read_csv("combined_dataset.csv")
X = data[['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']]
y = data['label']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.20, random_state=42, stratify=y
)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
print("RandomForest model trained successfully!")

```

Console output confirmed the split and successful training: data ready (488, 7), training data size (390, 7), testing data size (98, 7).

3.3 Step 5: Model Evaluation — Bharat

Once the trained model was pushed, it was evaluated on the held-out 20% test set (98 samples) using accuracy, a confusion matrix, and a per-class classification report.

```

from sklearn.metrics import classification_report, confusion_matrix

y_pred = model.predict(X_test)
print("Accuracy:", model.score(X_test, y_test))
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))

```

Results:

Class	Precision	Recall	F1-score	Support
Banana	1.00	1.00	1.00	28
Coconut	1.00	1.00	1.00	30
Papaya	1.00	1.00	1.00	20
Rice	1.00	1.00	1.00	20

Accuracy			1.00	98
Macro avg	1.00	1.00	1.00	98
Weighted avg	1.00	1.00	1.00	98

Confusion matrix (rows = actual, columns = predicted):

	Banana	Coconut	Papaya	Rice
Banana	28	0	0	0
Coconut	0	30	0	0
Papaya	0	0	20	0
Rice	0	0	0	20

Observation: the model scored 100% accuracy with zero misclassifications across all four crops — the confusion matrix is purely diagonal. This is a strong result, but a perfect score on a 4-class, well-separated dataset (only 488 rows total, 4 crops with visibly distinct N/P/K/rainfall ranges) is also a common symptom of an easy or small dataset rather than proof the model would generalize to a real farm-scale, many-crop setting. Worth flagging in the presentation rather than presenting as an unqualified win — cross-validation or testing against a held-out set from a different source would be the natural next check before adding more crops.

3.4 Step 6: Save Model + Build Interface — Sreedevi

The trained model, scaler, and feature list were bundled together (e.g., via joblib) so the same preprocessing is guaranteed at inference time. A Streamlit app was then built as the user-facing interface: numeric inputs for N, P, K, temperature, humidity, pH, and rainfall, with a button that returns the recommended crop.

```
import streamlit as st
import pandas as pd
import joblib

st.title("Crop Recommendation System")

bundle = joblib.load("crop_model.joblib")
model = bundle["model"]
scaler = bundle["scaler"]
features = bundle["features"]

N = st.number_input("Nitrogen (N)", value=60.0)
P = st.number_input("Phosphorus (P)", value=50.0)
```

```

K=st.number_input("Potassium (K)",value=42.0)
temperature=st.number_input("Temperature (° C)",value=28.0)
humidity=st.number_input("Humidity (%)",value=87.0)
ph=st.number_input("Soil pH",value=6.2)
rainfall=st.number_input("Rainfall (mm)",value=160.0)

if st.button("RecommendCrop"):
    input_df = pd.DataFrame([[N,P,K,temperature,humidity,ph,rainfall]],columns=features)
    input_scaled = scaler.transform(input_df)
    prediction = model.predict(input_scaled)[0]
    st.success(f"Recommended Crop: {prediction.capitalize()}")

```

3.5 Step 7: GitHub, Testing & Presentation — Aswani

Aswani's role covered three things that don't show up in model metrics but decide whether the project actually ships: reviewing and merging everyone's code into the shared repo, running the full pipeline end-to-end once all four pieces (preprocessing → training → evaluation → interface) were combined, and preparing the presentation slides.

4. Collaboration & Version Control Workflow

The original plan (Step 10 of the initial write-up) called for individual GitHub access with feature branches (data-preprocessing, model-training, frontend) merged via Pull Requests. Midway through, the team adjusted this: GitHub access was kept with the lead only, and other members sent their code over WhatsApp for the lead to integrate into the shared repository directly. The team explicitly agreed to run with a single common repository rather than per-member forks.

- Repo ownership: Project Lead (Aswani)
- Code handoff: WhatsApp → manually added to repo by the lead
- Integration & testing: Aswani, once all four modules were merged

5. Results Summary

- Dataset size: 488 rows across 4 crops (banana, coconut, papaya, rice), 7 numeric features.
- Train/test split: 390 training rows / 98 testing rows (80/20, stratified).
- Model: Random Forest Classifier, 100 estimators.
- Test accuracy: 100% (98/98 correct), precision/recall/F1 of 1.00 across all four classes.
- Deliverable: working Streamlit app that takes N, P, K, temperature, humidity, pH, and rainfall and returns a recommended crop.

6. Conclusion

The team successfully built an end-to-end crop recommendation pipeline — from raw Kaggle data through preprocessing, model training, evaluation, and a usable Streamlit interface — with the work cleanly divided across four members and one shared repository. The Random Forest model achieved perfect scores on the four-crop test set, validating the approach for this constrained scope.

7. Future Scope

- Expand beyond 4 crops to a larger, more realistic crop set to properly stress-test the model.
- Validate with cross-validation and/or an external test set to confirm the 100% accuracy isn't an artifact of a small, easily-separable dataset.
- Add feature importance analysis (which of N/P/K/temperature/humidity/pH/rainfall drives each prediction) since Random Forest supports this natively.
- Move to per-member GitHub access with feature branches and PR review, per the original Step 10 plan, once the team is comfortable with Git.
- Deploy the Streamlit app (e.g., Streamlit Community Cloud) so it's usable outside local machines.